Algorithm 1: `hex_to_bits(hex_str, width)`

函数作用: 完成十六进制与二进制位串之间的相互转换。前者把输入的 16 进制分组或密钥转为固定长度二进制串，便于后续按位处理。

Input: hex_str（十六进制字符串）；width（目标位宽）

Output: 指定长度的二进制串

// 先将十六进制字符串转成整数

value ← `int(hex_str, 16)`

// 利用格式化字符串补齐到指定位宽

return `f"{value:0{width}b}"`

Algorithm 2: `bits_to_hex(bits, width)`

函数作用: 完成二进制位串到十六进制字符串的转换。该函数用于把程序内部处理完的位串恢复为十六进制形式，便于输出与比对。

Input: bits（二进制字符串）；width（目标十六进制宽度）

Output: 指定长度的十六进制串

value ← `int(bits, 2)`

// 将二进制结果按十六进制格式输出

return `f"{value:0{width}X}"`

Algorithm 3: `permute(bits, table)`

函数作用: 提供 DES 中按表置换的基础操作。该函数用于 IP、IP^{-1}、E、P、PC-1、PC-2 等所有表驱动步骤。

Input: bits（位串）；table（置换表）

Output: 重排后的位串

// 置换函数

return ””.join(bits[i - 1] for i in table)

Algorithm 4: `xor_bits(a, b)`

函数作用: 提供 DES 中按位异或的基础操作。该函数用于子密钥与扩展结果的混合。

Input: a, b（等长位串）

Output: 异或结果位串

// 异或函数

return """.join('1' if x != y else '0' for x, y in zip(a, b))

Algorithm 5: `split_bits(bits, n)`

函数作用: 完成位串拆分。该函数用于把 56 bit 的密钥半部或 64 bit 的分组拆成左右两段，便于后续密钥调度或 Feistel 结构处理。

Input: bits（位串）；n（拆分位置）

Output: 拆分后的两个子串

// 左半部分保留前 n 位，右半部分保留剩余位

left $\leftarrow$ `bits[:n]`

right $\leftarrow$ `bits[n:]`

return (left, right)

Algorithm 6: `left_shift(bits, n)`

函数作用: 完成循环左移。该函数按照给定移位量对密钥半部进行循环左移，是 DES 密钥调度中的基础步骤。

Input: bits（位串）；n（左移位数）

Output: 左移后的位串

// 先对左移位数取模，避免超过位串长度

// 循环左移前先把位移量约束到合法范围

n $\leftarrow n \bmod |\texttt{bits}|$

return bits[n:] + bits[:n]

Algorithm 7: `generate_subkeys(key_hex)`

函数作用: 实现 DES 密钥调度。输入 64 bit 原始密钥后，先执行 PC-1 去掉校验位，再拆成左右两个 28 bit 半部，然后按照移位表循环左移，最后经过 PC-2 生成 16 个 48 bit 子密钥。

Input: key_hex（16 位十六进制密钥）

Output: 16 个子密钥组成的列表

key_bits ← `hex_to_bits(key_hex, 64)`

permuted ← `permute(key_bits, PC1)`

// PC-1 之后拆成左右两个 28 bit 半部

// 得到左右两个 28 bit 半部后，按轮次生成子密钥

(c, d) ← `split_bits(permuted, 28)`

subkeys ← [ ]

foreach shift ∈ `SHIFT_SCHEDULE` **do**

- c ← `left_shift(c, shift)`
- d ← `left_shift(d, shift)`
- subkey ← `permute(c + d, PC2)`
- 将 *subkey* 追加到 *subkeys*

return subkeys

Algorithm 8: `s_box_lookup(chunk6, sbox)`

函数作用: 完成单个 S 盒查表。每个 6 bit 子块的首尾两位组成行号，中间四位组成列号，查出对应 S 盒中的 4 bit 输出。

Input: chunk6（6 bit 子块）；sbox（单个 S 盒）

Output: 4 bit 输出

row ← `int(chunk6[0] + chunk6[5], 2)`

col ← `int(chunk6[1:5], 2)`

value ← `sbox[row][col]`

return `f"{value:04b}"`

Algorithm 9: `s_box_substitution(mixed)`

函数作用: 完成 8 个 S 盒的顺序代换。输入为 48 bit 混合结果，按 6 bit 切分后逐组查表，再拼接成 32 bit 输出。

Input: mixed（48 bit 输入）

Output: 32 bit 拼接结果

// 把 48 bit 输入切分为 8 个 6 bit 子块

chunks ← mixed 按 6 bit 切分为 8 组

substituted ← []

foreach chunk ∈ chunks **do**

    output ← `s_box_lookup(chunk, 对应 S 盒)`

    将 output 追加到 substituted

return `"".join(substituted)`

Algorithm 10: `round_function(r32, subkey48)`

函数作用: 实现 DES 轮函数 F。整体流程严格按照“E 扩展 → 异或 → S 盒代换 → P 置换”的顺序执行，输出 32 bit 轮函数结果。

Input: r32（32 bit 右半部分）；subkey48（48 bit 子密钥）

Output: 32 bit 轮函数输出

// 轮函数依次执行扩展、异或、S 盒替代和 P 置换

expanded ← `permute(r32, E)`

mixed ← `xor_bits(expanded, subkey48)`

substituted ← `s_box_substitution(mixed)`

return `permute(substituted, P)`

Algorithm 11: `des_encrypt_block(plaintext_hex, key_hex)`

函数作用: 完成单个 64 bit 数据分组的 DES 加密。流程依次为：输入转换、初始置换 IP、16 轮 Feistel 迭代、左右交换、逆初始置换 IP^{-1}，最终输出密文。

Input: plaintext_hex（明文分组）；key_hex（主密钥）

Output: 密文十六进制串；可选返回每轮调试信息

block ← `hex_to_bits(plaintext_hex, 64)`

subkeys ← `generate_subkeys(key_hex)`

// 先做初始置换，再拆成左右两半

state ← `permute(block, IP)`

(l, r) ← `split_bits(state, 32)`

rounds ← [ ]

foreach subkey ∈ subkeys **do**

  new_l ← r

  new_r ← `xor_bits(l, round_function(r, subkey))`

  l, r ← new_l, new_r

  将 (l, r) 记录到 rounds

// 左右交换后再做逆初始置换

// 16 轮结束后交换左右半部，再做逆初始置换

preoutput ← r + l

ciphertext ← `bits_to_hex(permute(preoutput, IP_INV), 16)`

return (ciphertext, rounds)

Algorithm 12: `des_decrypt_block(ciphertext_hex, key_hex)`

函数作用: 完成单个 64 bit 数据分组的 DES 解密。解密流程与加密流程一致，只是把子密钥顺序反转后再送入同样的 Feistel 迭代框架。

Input: ciphertext_hex（密文分组）；key_hex（主密钥）

Output: 明文十六进制串；可选返回每轮调试信息

block ← `hex_to_bits(ciphertext_hex, 64)`

subkeys ← `reverse(generate_subkeys(key_hex))`

// 解密流程与加密一致，只是子密钥顺序反转

state ← `permute(block, IP)`

(l, r) ← `split_bits(state, 32)`

rounds ← [ ]

foreach subkey ∈ subkeys **do**

  new_l ← r

  new_r ← `xor_bits(l, round_function(r, subkey))`

  l, r ← new_l, new_r

  将 (l, r) 记录到 rounds

// 解密与加密结构相同，只是子密钥顺序相反

preoutput ← r + l

plaintext ← `bits_to_hex(permute(preoutput, IP_INV), 16)`

return (plaintext, rounds)

Algorithm 13: `main.py` 的调用与输出流程

函数作用: 主控文件不直接参与 DES 计算，而是负责解析命令行参数、调用加密或解密函数、打印结果并记录运行时间。默认情况下会运行演示模式，自动验证预置测试向量。

Input: 用户输入的密钥、明文或密文，以及运行模式参数

Output: 终端输出或 txt 实验报告内容

// 读取输入参数并判断运行模式

解析命令行参数

if 未指定任何攻击或测试模式 **then**

    进入演示模式，依次运行预置测试用例

else

    根据用户输入调用加密或解密流程

// 输出密文、明文、调试状态和时间信息

打印结果并写入实验报告文本